Windows における ROS2 環境構築手順 (WSL2 + Docker Network Mirroring)

January 1, 2026

1 背景と目的

Windows 上で ROS2 環境を構築する際、Docker Desktop がインストールされていると、WSL 側で Docker を操作しようとした際に Docker Desktop が自動的に介入する場合がある。

この挙動により、WSL2 側で意図したネットワーク設定が反映されず、ROS2 の通信が確立できないというトラブルが頻発する。WSL で完結させたいにも関わらず、Windows 側のアプリケーションが干渉し、設定がうまくいかない状況は非常に煩わしく、開発における大きなストレス要因（イライラ）となっている。

そこで本ドキュメントでは、この諸悪の根源となりうる Docker Desktop をアンインストールし、WSL2 のネイティブな環境とネットワークミラーリング機能を用いて、クリーンで快適な ROS2 実行環境を構築する手順を示す。

2 手順

2.1 Docker Desktop のアンインストール

Windows に既に諸悪の根源「Docker Desktop」がインストールされている場合は、競合を避けるためにアンインストールを行う。

1. Windows の「設定」>「アプリ」>「インストールされているアプリ」を開く。
2. Docker Desktop を探し、アンインストールを実行する。

2.2 WSL2 の有効化

以下の記事を参考に、Windows Subsystem for Linux 2 (WSL2) を有効化し、Ubuntu などのディストリビューションをインストールする。

参考記事: Qiita - WSL2 の導入と設定
https://qiita.com/tatsushi_do_ob/items/a2b018a7eaaa84670d91

2.3 ネットワーク設定 (.wslconfig)

ROS2 の通信（DDS）を円滑に行うため、WSL2 のネットワークモードを「ミラーリング」に設定する。

1. Windows のユーザーホームディレクトリ（例: `C:\Users\ユーザー名`）に移動する。
2. `.wslconfig` というファイルを作成する（既にある場合は編集する）。
3. 以下の内容を記述して保存する。

```
[wsl2]
networkingMode=mirrored
```

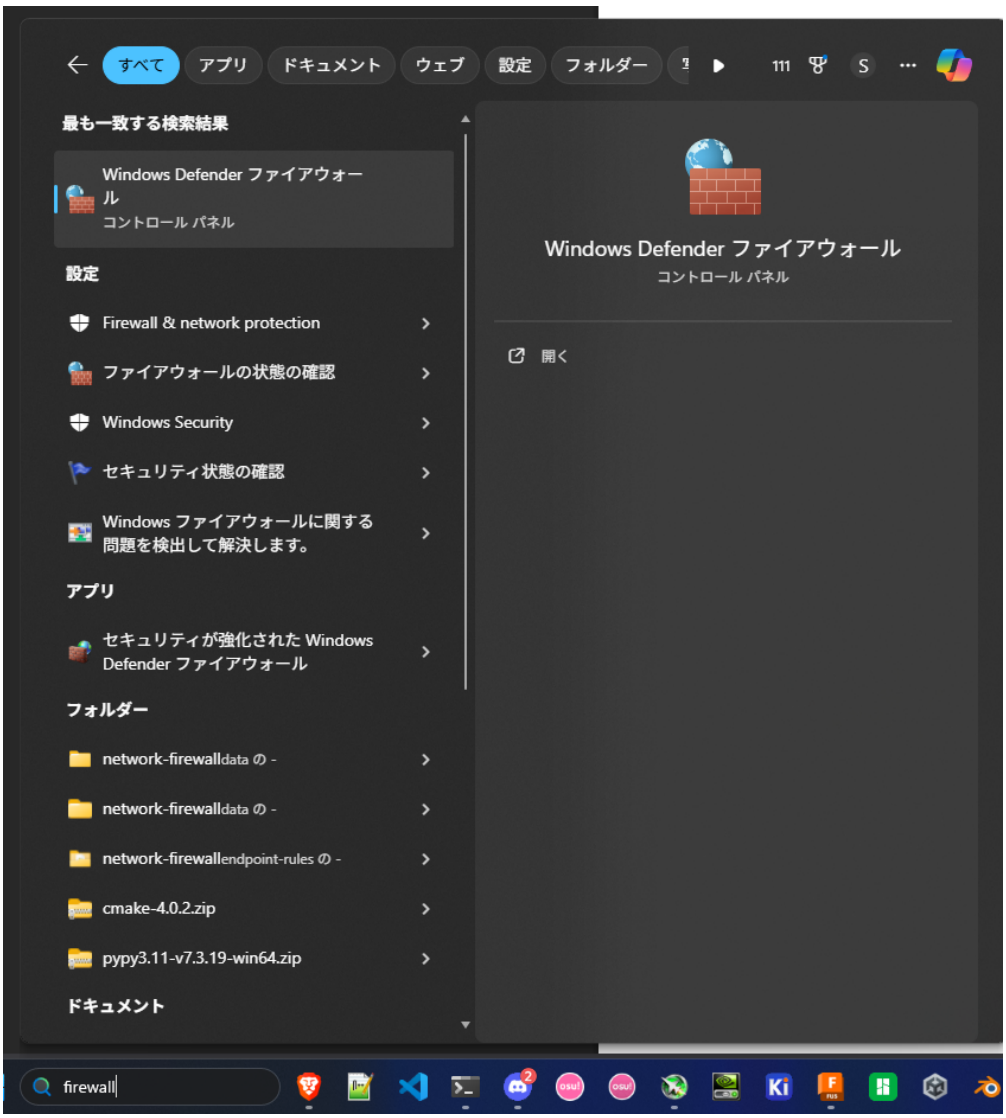
2.4 Windows ファイアウォールの無効化

通信の遮断を防ぐため、Windows 側のファイアウォールを無効化する。以下の図を参考に設定を行う。

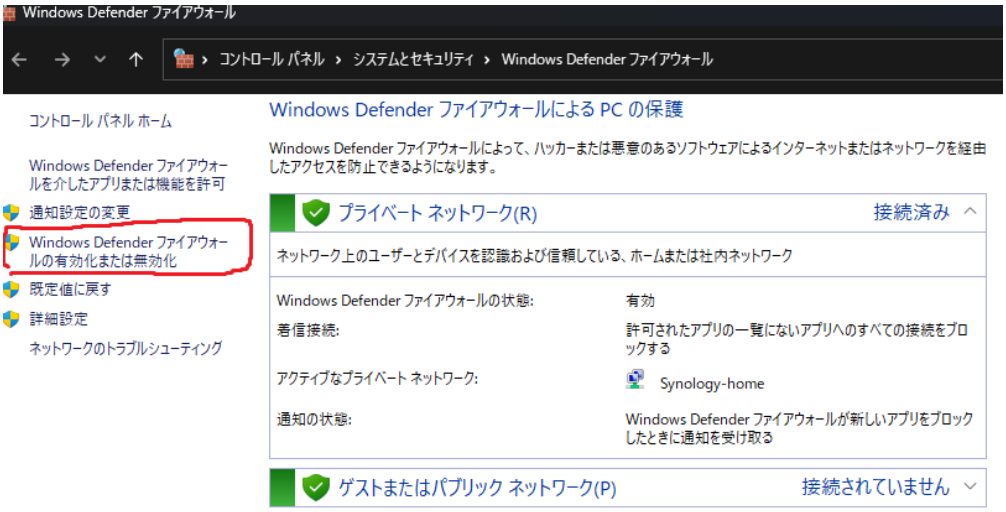
1. 「プライベートネットワーク」のファイアウォールを**無効**にする。
2. 「パブリックネットワーク」のファイアウォールを**無効**にする。

※セキュリティリスクがあるため、信頼できるネットワーク環境下で行うこと。

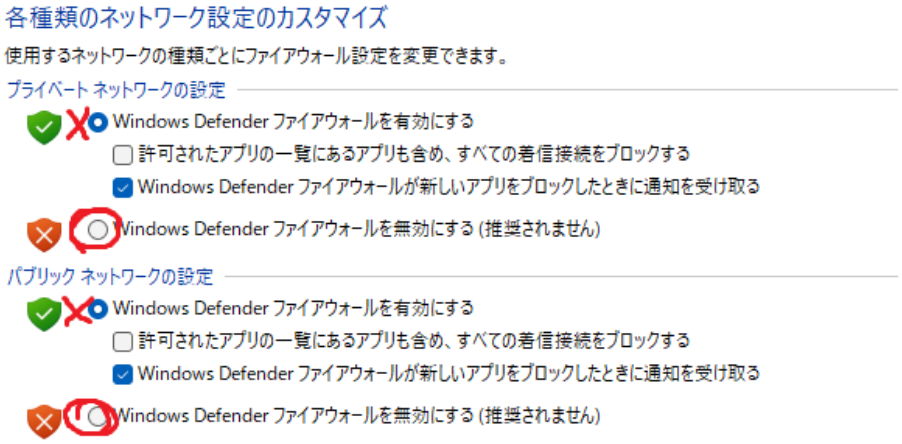
まず、firewall と検索して一番上に出てくる Windows Defender ファイアフォールを開く。



次に開いた画面の左上側の選択肢の中から Windows Defender ファイアウォールの有効化または無効化を開く。



最後に、各種類のネットワーク設定のカスタマイズからプライベートネットワークとパブリックネットワークの Windows Defender ファイアウォールを有効にする から Windows Defender ファイアウォールを無効にする (推奨されません) に変更する。



2.5 WSL の再起動

設定を反映させるため、PowerShell またはコマンドプロンプトで以下のコマンドを実行し、WSL を再起動する。

```
wsl --shutdown
```

その後、WSL（Ubuntu など）を再度起動する。

2.6 Docker のインストール (apt リポジトリの利用)

WSL2 内部（Ubuntu）に Docker エンジンをインストールする。公式ドキュメントに基づき、以下の手順を実行する。

2.6.1 競合パッケージの削除

非公式パッケージや古いバージョンがインストールされている場合は、競合を避けるために削除する。

```
for pkg in docker.io docker-doc docker-compose docker-compose-v2 podman
   -docker containerd runc; do sudo apt-get remove $pkg; done
```

2.6.2 apt リポジトリのセットアップ

Docker の公式リポジトリを追加するための準備を行う。

まず、必要なパッケージをインストールし、GPG キーを追加する。

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/
   apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

次に、リポジトリを Apt ソースに追加する。

```
# Add the repository to Apt sources:
echo \
  "deb␣[arch=$(dpkg␣--print-architecture)␣signed-by=/etc/apt/keyrings/
     docker.asc]␣https://download.docker.com/linux/ubuntu␣\
␣␣$(.␣/etc/os-release␣&&␣echo␣"$VERSION_CODENAME")␣stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
```

2.6.3 Docker Engine のインストール

最新バージョンの Docker Engine、CLI、およびプラグインをインストールする。

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-
   plugin docker-compose-plugin
```

2.6.4 動作確認

インストールが完了したら、以下のコマンドでDockerサービスの状態を確認し、hello-worldイメージを実行して正しく動作するか検証する。

```
# サービスの確認
sudo systemctl status docker

# hello-worldの実行
sudo docker run hello-world
```

3 まとめ

以上で環境構築は完了である。これにより、Docker Desktopの干渉に悩まされることなく、Windows上のWSL環境にてDockerを用いたROS2開発が可能となる。